

3 Normalizing Flows

An important goal in statistical modeling is to sample from a desired target probability distribution $p_{\text{target}}(x)$. Unfortunately, very often we are not able to do so directly, because the probability density of interest is not tractable. This is usually due to high dimensional integrals, which appear in the calculation of the normalization constant. To circumvent this problem, we can make use of Monte Carlo methods like the Metropolis-Hastings algorithm, where knowing a function proportional to p_{target} is sufficient. However, Monte Carlo sampling can be computationally quite expensive. A more efficient approach is to utilize generative ML models. One of the first network structures with the ability to capture a probability density are Boltzmann machines. The binary nodes of this type of memory model, a so-called Hopfield network, behave exactly the same way like spins in an Ising model. This means, if we choose the energy of the Ising model as the loss function, we are effectively searching for an equilibrium state during training. The configuration of the units will then follow the Boltzmann distribution $p \sim e^{-E}$. Although samples from this density can be easily obtained by taking random configurations of the nodes, the underlying distribution function is not accessible. This drawback is also present in other more efficient ML models like autoencoders or GANs.

A solution is provided by normalizing flows [2]. These neural networks can evaluate the learned distribution exactly, because it is obtained by the application of a series of invertible transformations on a simple base distribution, typically taken to be Gaussian. The flow network effectively performs a change of variables on normal distributions. Since it is invertible, the reverse flow map can also be used. Training the network then equates to finding a set of maps, which transform a complicated probability density into the product of normal distributions, i.e. the network normalizes the target distribution. A detailed review of normalizing flows is [1], which we will largely follow.

We will present basic concepts for flow networks in the case of complex variables. This is a bit unusual, because neural networks are usually build with real variables and complex numbers are defined as 2D vectors. But there has been great effort recently to implement complex numbers natively into the standard ML-libraries.

In this chapter we will denote the data vector by $x = (x^1, \dots, x^N)^T \in \mathbb{C}^N$ and the (latent) variable of the base distribution by $z = (z^1, \dots, z^N)^T \in \mathbb{C}^N$, which is common in the ML literature.

3.1 Tractable probability density

In the ideal case, the network transformation G^{-1} maps the input data x , which follows the target distribution $p_{\text{target}}(x)$, to the latent random variables $G^{-1}(x) = z \sim p_{\text{base}}(z)$. Conversely, we can also sample from the base distribution $p_{\text{base}}(z)$ by applying the inverse map on z to obtain new data $x = G(z)$. The two distributions are related by a change of variables with the Jacobian matrix

$$J_G(z) = \begin{pmatrix} \frac{\partial G^1}{\partial z^1} & \cdots & \frac{\partial G^1}{\partial z^N} \\ \vdots & \ddots & \vdots \\ \frac{\partial G^N}{\partial z^1} & \cdots & \frac{\partial G^N}{\partial z^N} \end{pmatrix}. \quad (3.1)$$

Because we are using complex numbers and $\mathbb{C} \cong \mathbb{R}^2$, the vector x actually represents $2N$ independent variables. So when we change from x to z , we have to account for contributions coming from x and also its conjugate $x^\dagger$. If our flow treats the real and imaginary part of x and z in the same way, i.e. they transform under the same map, then $\det J_{G^{-1}}^*(x) = \det J_{G^{-1}}(x^\dagger)$ and the target density can then be rewritten as

$$\begin{aligned} p_{\text{flow}}(x) &= p_{\text{base}}(G^{-1}(x)) \left| \det \frac{\partial G^{-1}}{\partial x} \right| \left| \det \frac{\partial G^{-1}}{\partial x^\dagger} \right| \\ &= p_{\text{base}}(z) \left| \det \frac{\partial G}{\partial z} \right|^{-1} \left| \det \frac{\partial G}{\partial z^\dagger} \right|^{-1} \\ &= p_{\text{base}}(z) |\det J_G(z)|^{-2}, \end{aligned} \quad (3.2)$$

where the switch from G^{-1} to G is allowed by the inverse function theorem. When we choose a suitable base distribution, we can take almost any density, even a very restricted one like the uniform distribution. In fact, if $p_{\text{target}}(x)$ and $p_{\text{base}}(z)$ are well-behaved, there always exist a diffeomorphism to transform one to the other (see [1] for a proof).

The extension to multiple bijective maps G_k is straightforward and done by simple composition of the transformations, see figure 3.1. The full map of the flow consisting of K

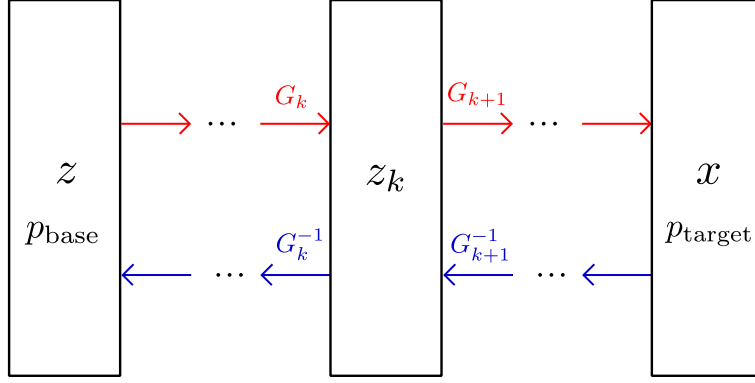


Figure 3.1: Flow network consisting of multiple bijective maps

invertible layers is then given by

$$G = G_K \circ \dots \circ G_1 \quad (3.3)$$

with inverse transformation

$$G^{-1} = G_1^{-1} \circ \dots \circ G_K^{-1} . \quad (3.4)$$

The Jacobian determinant for G is still easily accessible as a product of each layer:

$$\begin{aligned} \det J_G(z) &= \det J_{G_K}(G_{K-1}(z_{K-2})) \cdots \det J_{G_1}(z) \\ &= \prod_{k=1}^K \det J_{G_k}(z_{k-1}) , \end{aligned} \quad (3.5)$$

where we defined $z_k = G_k(z_{k-1})$ and identified $z_0 \equiv z$, $z_K \equiv x$. By collecting the Jacobian at each layer during a pass through the flow network, one can easily obtain the determinant of the full transformation. Very often, we will not directly handle the determinants, but instead calculate the logarithm of them. This is because in practice one usually works with log probabilities, as they are additive and therefore easier to manipulate. This is also favorable from the computational point of view, because determinants can easily blow up in value for high dimensional data. Calculating the logarithm of Jacobians is more stable. For the expression (3.5) we get

$$\log |\det J_G(z)| = \sum_{k=1}^K \log |\det J_{G_k}(z_{k-1})| , \quad (3.6)$$

which is now a summation over the layer contributions instead of a product.

3.2 Unitary flows

The process of designing the transformation G_k is usually guided by the premises to have tractable Jacobians and inverse map. Ideally, the log determinants are easy to compute and the inverse map has a simple form. But the only necessary requirement is the existence of the inverse or a non-zero Jacobian. One could implement G_k as entire neural networks, which is the case for flows like RNVPs. In the simplest case, G_k is just a linear layer. By constraining the weight matrix to be orthogonal or unitary, we get a trivial Jacobian and $\log |\det J| = 0$. The computation of the inverse for unitary matrices also requires no special solving algorithm. We will discuss two ways to transform a weight matrix $\tilde{M}$ into a unitary layer U .

Assuming we have a upper triangular matrix with zeros on the diagonal of the form

$$\tilde{M} = \begin{pmatrix} 0 & m_{1,2} & \dots & m_{1,n} \\ 0 & 0 & \ddots & \vdots \\ \vdots & & \ddots & m_{n-1,n} \\ 0 & 0 & \dots & 0 \end{pmatrix}. \quad (3.7)$$

$\tilde{M}$ can be made into a skew-hermitian (or anti-hermitian) matrix with

$$M = \tilde{M} - \tilde{M}^\dagger. \quad (3.8)$$

Since the space of skew-hermitian matrices with zero trace form the Lie algebra of the special unitary group $SU(n)$, we can apply the exponential map on M and get a unitary layer U for the flow:

$$U = e^M. \quad (3.9)$$

Another possibility to construct a unitary matrix from the anti-hermitian M is to use the so-called Cayley transform, which is defined as

$$U = (I - M)^{-1}(I + M), \quad (3.10)$$

where I is the identity matrix. The inverse transformation is given by

$$M = (U - I)(U + I)^{-1}. \quad (3.11)$$

One should note, that we cannot generate any element of $SU(n)$ by this transformation, since we are excluding certain matrices, for example those which are unitary but not skew-

hermitian. Both options, the Cayley and exponential map, take $\mathcal{O}(n^3)$ time to compute, which will become inefficient for large matrices.

3.3 Kullback-Leibler divergence

In order to train a flow network for an inference or modeling task, an appropriate choice for the loss function has to be made. The most common choice is the relative entropy or Kullback-Leibler divergence. It serves as a measure for how much two distributions differ from each other. It is important to know that the divergence is not symmetric in its arguments. For flow networks we can distinguish between the forward KL divergence and the reversed one.

The forward version is defined as

$$\begin{aligned} D_{\text{KL}}(p_{\text{target}}(x) \parallel p_{\text{flow}}(x)) &= \sum_x p_{\text{target}}(x) \log \left(\frac{p_{\text{target}}(x)}{p_{\text{flow}}(x)} \right) \\ &= \mathbb{E}_{x \sim p_{\text{target}}} [\log p_{\text{target}} - \log p_{\text{flow}}] \\ &= \mathbb{E}_{x \sim p_{\text{target}}} [\log p_{\text{target}} - \log |\det J_{G^{-1}}| - \log p_{\text{base}}] , \end{aligned} \quad (3.12)$$

where the expectation value over the target distribution is carried out as an averaging over the training set. The KL-divergence is zero, if the two distributions are equal. The divergence can be interpreted as the additional entropy, if one uses the distribution p_{flow} instead of the target p_{target} to sample x . The expectation over p_{target} will give us a constant term, such that the loss function can be defined as

$$\mathcal{L} = \mathbb{E}_{x \sim p_{\text{target}}} [\log |\det J_G| - \log p_{\text{base}}] \quad (3.13)$$

It is worth noting that in this approach we do not need to calculate $G(z)$, although it is of course required for sampling. In the holographic network, which will be presented in the next chapter, we do not have a training set at hand. The evaluation of the expectation value is therefore not possible. However, we can use the relative entropy function with reversed arguments, such that the loss includes an expectation value over the network distribution, i.e. the averaging over a sampled data set.

The reversed KL divergence is more suitable for our purposes and is defined as

$$\begin{aligned} D_{\text{KL}}(p_{\text{flow}}(x) \parallel p_{\text{target}}(x)) &= \mathbb{E}_{x \sim p_{\text{flow}}} [\log p_{\text{flow}} - \log p_{\text{target}}] \\ &= \mathbb{E}_z [\log p_{\text{base}} - \log |\det J_G| - \log p_{\text{target}}] . \end{aligned} \quad (3.14)$$

For the evaluation of the expectation value of p_{target} , a function proportional to the target distribution $\tilde{p}_{\text{target}} \sim p_{\text{target}}$ is sufficient, because the normalization constant will give a constant term due to the logarithm. Our loss function can then be chosen to be

$$\mathcal{L} = \mathbb{E}_z [\log p_{\text{base}} - \log |\det J_G| - \log \tilde{p}_{\text{target}}] \quad (3.15)$$

Training of the flow under reversed KL-divergence means that we are trying to find an approximation of p_{target} by fitting a function p_{flow} to the target distribution. This process is called probability density distillation.

There is an interesting connection between the training procedures under the two versions of the KL-divergence. Let us first define the probability distribution $p_{\text{target}}^*(z)$, which is the density induced by the network transformation via $p_{\text{target}} \xrightarrow{G^{-1}} p_{\text{target}}^*$. It can be found by changing variables from x to z for $p_{\text{target}}(x)$:

$$p_{\text{target}}^*(z) = p_{\text{target}}(x) |\det J_G|^2. \quad (3.16)$$

A forward KL-divergence can be obtained from the reversed version by using the above transformation:

$$\begin{aligned} D_{\text{KL}}(p_{\text{flow}} \parallel p_{\text{target}}) &= \mathbb{E}_z [\log p_{\text{base}}(z) - \log |\det J_G(z)| - \log p_{\text{target}}(G(z))] \\ &= \mathbb{E}_z [\log p_{\text{base}}(z) - \log p_{\text{target}}^*(z)] \\ &= D_{\text{KL}}(p_{\text{base}}(z) \parallel p_{\text{target}}^*(z)). \end{aligned} \quad (3.17)$$

We see that fitting the distribution $p_{\text{flow}}(x)$ to the target $p_{\text{target}}(x)$ under the reversed KL-divergence is equivalent to fitting the induced density $p_{\text{target}}^*(z)$ to the base distribution under the forward divergence.

3.4 Reparametrization trick

The typical choice for the base density is a normal distribution. In the simplest case each variable $z^\alpha \in \mathbb{C}$ is drawn from $\mathcal{N}_{\mathbb{C}}(0, 1)$ with density

$$p(z^\alpha) = \frac{1}{\pi} e^{-|z^\alpha|^2}. \quad (3.18)$$

This means, that all variables are i.i.d. and have no correlation among them. A less biased choice is a multivariate complex Gaussian

$$p_{\text{base}}(z) = \frac{1}{\det(\pi\Sigma)} \exp\left[-\frac{1}{2}z^\dagger \Sigma^{-1} z\right], \quad (3.19)$$

where Σ is called the covariance matrix. Its off-diagonal components contain the correlation between different random variables z^α . This distribution has circularly symmetry, which means, that z is evenly distributed across all directions in the complex plane.

The matrix Σ can be made learnable via the reparametrization trick. Since the evaluation of the loss function involve the expectation value over the base distribution, it is difficult to parameters contained in p_{base} . The idea is to implement the multivariate Gaussian as a linear layer. We observe, that we can obtain $z \sim p_{\text{base}}(z)$ from a transformation of i.i.d. variables $\epsilon \sim \mathcal{N}_{\mathbb{C}(0,1)}$ via $z = L\epsilon$. The lower triangluar matrix L can be calculated from the Cholesky decomposition of the covariance and satisfies $\Sigma = LL^\dagger$. We can make a quick check by changing variable for the base distribution to get

$$p_{\text{base}}(\epsilon) = \frac{1}{\pi^N} \exp\left[-\frac{1}{2}\epsilon^\dagger \epsilon\right], \quad (3.20)$$

where all correlation between the random variables have been removed.

Bibliography

- [1] G. Papamakarios et al. “Normalizing Flows for Probabilistic Modeling and Inference”. In: *Journal of Machine Learning Research*, 22(57):1-64, 2021 (2019). arXiv: 1912.02762 [stat.ML].
- [2] D. J. Rezende and S. Mohamed. “Variational Inference with Normalizing Flows”. In: *Proceedings of Machine Learning Research* 37 (2015). arXiv: 1505.05770 [stat.ML].